

MemLab 16 MiB

Informe final del proyecto

Simulador educativo de gestión de memoria principal

Fecha de evaluación: 21 de septiembre de 2026

Commit base: 20cec7a

Estado evaluado: copia de trabajo local actual

Tecnologías: HTML5, CSS y JavaScript

Índice

1. Resumen ejecutivo	3
2. Contexto y problema abordado	3
3. Objetivos del proyecto	3
3.1. Objetivo general	3
3.2. Objetivos específicos observados	3
4. Alcance realizado	3
5. Solución construida	4
5.1. Estructura	4
5.2. Arquitectura funcional	5
5.3. Modelo de memoria	5
6. Algoritmos implementados	5
6.1. Asignación	5
6.2. Liberación y compactación	5
6.3. Indicadores	5
7. Experiencia de usuario	5
8. Metodología de evaluación	6
9. Resultados de validación	6
10. Evaluación de calidad	7
11. Hallazgos, riesgos y limitaciones	7
11.1. Prioridad alta	7
11.2. Prioridad media	7
11.3. Prioridad baja o pedagógica	7
12. Entregables documentales	8
13. Recomendaciones	8
13.1. Corto plazo	8
13.2. Mediano plazo	8
13.3. Extensiones académicas	8
14. Integridad del código fuente	9

15. Conclusión final

9

1 Resumen ejecutivo

Resumen ejecutivo

MemLab 16 MiB cumple su propósito central como herramienta didáctica para explorar asignación contigua de memoria. La aplicación implementa tres esquemas de particionado, tres algoritmos de ajuste, liberación y coalescencia de huecos, compactación manual/automática, métricas y trazabilidad mediante bitácora. Su arquitectura estática reduce las barreras de ejecución y despliegue. La evaluación detectó una exposición de inserción HTML en la bitácora, algunos casos límite de mensajería/configuración y ausencia de pruebas automatizadas; son oportunidades de mejora, no impedimentos para el uso educativo local controlado.

2 Contexto y problema abordado

Los conceptos de fragmentación y selección de particiones suelen presentarse como diagramas estáticos. El proyecto transforma esos conceptos en un laboratorio interactivo: el estudiante modifica límites del sistema operativo, particiones, políticas y trabajos, y observa cómo cambian direcciones, utilización, desperdicio y admisiones.

El espacio simulado contiene 16 MiB, equivalentes a 2^{24} bytes. La porción inicial pertenece al sistema operativo y el resto se administra mediante:

1. particiones estáticas iguales;
2. particiones estáticas de tamaños variables;
3. particiones dinámicas con fusión y compactación.

3 Objetivos del proyecto

3.1 Objetivo general

Proporcionar una aplicación web comprensible y manipulable que permita visualizar la asignación contigua de memoria y contrastar sus algoritmos y formas de fragmentación.

3.2 Objetivos específicos observados

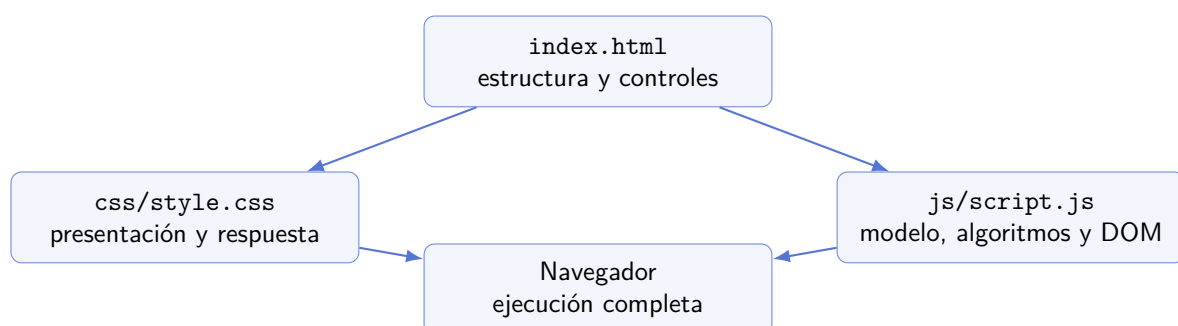
- representar direcciones físicas y tamaños en unidades binarias;
- modelar procesos con segmentos de código, datos, pila y heap;
- aplicar primer, mejor y peor ajuste donde corresponda;
- mostrar fragmentación interna y externa mediante mapa y KPI;
- permitir liberación, fusión de huecos y compactación;
- explicar cada decisión relevante en una bitácora;
- ejecutarse sin backend ni construcción previa;
- adaptarse a tema del sistema y pantallas estrechas.

4 Alcance realizado

Capacidad	Estado	Implementación observada
Espacio de 16 MiB	Implementado	Constantes binarias y direcciones hexadecimales de 24 bits.
Reserva de S.O.	Implementado	Opciones de 1, 2 y 4 MiB al inicio del mapa.
Particiones fijas	Implementado	2 a 16 particiones según opciones, redondeo a 4 KiB.
Particiones variables	Implementado	Tabla editable en KiB y zona no particionada.
Particiones dinámicas	Implementado	División exacta de huecos y cero fragmentación interna.
Tres algoritmos	Implementado	Primer, mejor y peor ajuste por recorrido lineal.
Liberación y fusión	Implementado	Coalescencia de huecos dinámicos contiguos.
Compactación	Implementado	Manual y automática al fallar por discontinuidad.
Métricas	Implementado	Utilización, dos fragmentaciones, mayor bloque y procesos.
Programas personalizados	Implementado	Nombre y cuatro segmentos en KiB.
Escenario pedagógico	Implementado	Carga y liberación alternada para producir huecos.
Persistencia	Fuera del alcance actual	Todo el estado se pierde al recargar.
Backend / cuentas	Fuera del alcance actual	La aplicación es completamente estática.
Suite de pruebas	No incluida	No hay infraestructura ni casos automatizados en el repositorio.

5 Solución construida

5.1 Estructura



El README completa el conjunto con instrucciones y descripción conceptual. No hay archivos de configuración, dependencias de aplicación ni compilación.

5.2 Arquitectura funcional

El JavaScript está encapsulado en una IIFE con estado mutable privado. Los eventos DOM llaman reglas de negocio sincrónicas; cada cambio termina en un renderizado integral o parcial. La interfaz se deriva del estado, por lo que el mapa, los KPI, la tabla y la bitácora se mantienen coordinados.

La decisión de regenerar `innerHTML` es suficiente para el volumen actual y evita complejidad de framework. A cambio, el dominio queda acoplado al DOM y es más difícil probarlo de forma aislada.

5.3 Modelo de memoria

El arreglo `mem` mantiene bloques ordenados con base, tamaño, tipo y PID. Los procesos incluyen cuatro segmentos, tamaño agregado, estado y ubicación. La configuración activa define método, algoritmo, reserva del S.O., particiones y compactación automática.

La configuración inicial reserva 2 MiB y crea siete particiones de 2 MiB. De los ocho trabajos, cuatro se cargan durante el arranque. El catálogo total requiere 18 880 KiB y la memoria de usuario predeterminada ofrece 14 336 KiB, lo que garantiza decisiones visibles de admisión y rechazo.

6 Algoritmos implementados

6.1 Asignación

El proceso general filtra bloques libres con capacidad suficiente y selecciona uno:

- primer ajuste: primer candidato en orden de dirección;
- mejor ajuste: candidato de menor capacidad válida;
- peor ajuste: candidato de mayor capacidad válida.

En particiones fijas iguales se fuerza la primera disponible. En dinámica, el hueco se divide para que el bloque ocupado tenga exactamente el tamaño del proceso.

6.2 Liberación y compactación

Una liberación estática limpia el PID, sin alterar límites. Una liberación dinámica convierte el bloque en hueco y combina vecinos. La compactación ordena los procesos por dirección y los desplaza hacia el final del S.O.; todo el espacio disponible queda, si existe, en un hueco final.

La compactación automática solo se intenta cuando el total libre es suficiente pero ningún hueco individual alcanza. Este criterio distingue correctamente falta de capacidad total de falta de continuidad.

6.3 Indicadores

La utilización se calcula sobre memoria de usuario y solo suma tamaños útiles de procesos. La fragmentación interna suma sobrantes dentro de bloques ocupados. En dinámica, la fragmentación externa se define como libre total menos el hueco más grande. En estáticas, el proyecto contabiliza como externa toda partición libre no combinable, una convención pedagógica documentada en la interfaz.

7 Experiencia de usuario

La página organiza la experiencia en cinco niveles:

1. método y acciones globales;
2. configuración y algoritmo;
3. KPI de lectura inmediata;
4. mapa, detalle y tabla de trabajos;
5. bitácora explicativa.

El mapa usa altura proporcional al tamaño real y adapta la cantidad de texto cuando un bloque es pequeño. Los colores diferencian procesos y segmentos; los patrones identifican fragmentación y espacio reservado. En pantallas de hasta 900 px, la disposición pasa a una columna.

La interfaz incorpora etiquetas ARIA, estados `aria-pressed`, foco visible, región activa para la bitácora, tema oscuro y consideración de movimiento reducido.

8 Metodología de evaluación

La revisión final fue deliberadamente no invasiva:

1. inventario completo del repositorio y lectura de sus cuatro archivos;
2. trazado de las 33 funciones declaradas y sus efectos sobre el estado;
3. revisión de fórmulas, rangos, estados y eventos;
4. comprobación sintáctica con Node.js;
5. servicio temporal por HTTP y solicitud de HTML, CSS y JS;
6. comparación automática de selectores `#id` frente al HTML;
7. cálculo independiente de tamaños de catálogo y configuración predeterminada;
8. registro de huellas SHA-256 de los archivos originales para verificar que la generación documental no los modificara.

Se intentó preparar una validación con navegador automatizado conforme a la guía de pruebas disponible, pero el entorno no contiene la dependencia Python Playwright. No se instaló en el proyecto para respetar el alcance de “solo entregables”.

9 Resultados de validación

Comprobación	Resultado	Detalle
Sintaxis JavaScript	Aprobada	Node.js no reportó errores de análisis.
Disponibilidad HTTP	Aprobada	Los tres recursos ejecutables respondieron 200.
Tipos MIME	Aprobada	HTML, CSS y JavaScript fueron servidos con tipos correspondientes.
Integridad de IDs	Aprobada	33 IDs únicos y ningún selector auxiliar faltante.
Catálogo de programas	Verificado	Totales recalculados a partir de los cuatro segmentos.
Tamaño fijo inicial	Verificado	14 MiB / 7 = 2 MiB por partición.
Tabla variable inicial	Verificada	Los seis tamaños suman 14 336 KiB.
Prueba end-to-end real	Pendiente	No disponible Playwright en el entorno.

10 Evaluación de calidad

Atributo	Valoración	Fundamento
Adecuación funcional	Alta	Cubre métodos, ajustes, métricas, escenarios y explicación de decisiones.
Comprensibilidad	Alta	Código comentado, nombres descriptivos y bitácora pedagógica.
Portabilidad	Alta	Archivos estáticos sin construcción ni backend.
Rendimiento	Adecuado	Recorridos lineales y render total sobre conjuntos pequeños.
Mantenibilidad	Media-alta	Separación por funciones, aunque modelo y DOM comparten un único módulo.
Accesibilidad	Media-alta	Buenas bases ARIA y CSS; falta verificación formal con herramientas/lectores.
Seguridad	Media	Superficie pequeña, pero existe inserción HTML en la bitácora.
Testabilidad	Media-baja	No hay pruebas y el estado privado está acoplado al navegador.
Observabilidad	Alta para docencia	La bitácora explica causas y efectos hasta 80 eventos.

11 Hallazgos, riesgos y limitaciones

11.1 Prioridad alta

Riesgo de XSS DOM en nombres personalizados. El nombre se escapa al mostrarlo en mapa, detalle y tabla, pero se concatena sin escape a un mensaje de la bitácora, que luego se inserta mediante `innerHTML`. El riesgo es limitado en uso local individual, pero debe corregirse antes de aceptar datos no confiables o publicar la aplicación en un entorno compartido.

11.2 Prioridad media

- al aumentar el tamaño del S.O., la tabla variable puede sumar más que la memoria disponible; el constructor omite desde la primera partición que no cabe, pero el texto de suma sigue considerando toda la configuración;
- algunos mensajes de rechazo del modo fijo pueden atribuir el fallo a ocupación total cuando existen particiones libres pero demasiado pequeñas;
- compactar una memoria totalmente llena puede registrar “1 hueco de 0 KiB” aunque no se agregue un bloque de hueco;
- no existe persistencia ni exportación de escenario/bitácora;
- no hay pruebas automatizadas de regresión.

11.3 Prioridad baja o pedagógica

- la denominación de fragmentación externa en esquemas estáticos es una simplificación didáctica que debe explicarse al compararla con otras fuentes;
- `color-mix` puede degradar la apariencia en navegadores antiguos;

- la lista de eventos se limita a 80 y no representa tiempo real;
- el sistema simula asignación contigua y no cubre paginación, segmentación no contigua, memoria virtual, swapping ni traducción mediante MMU.

12 Entregables documentales

Como parte del cierre se produjeron, sin modificar el código fuente:

Entregable	Contenido
Manual técnico (LaTeX y PDF)	Arquitectura, datos, algoritmos, fórmulas, catálogo de funciones, eventos, ejecución, despliegue, validación, riesgos y mantenimiento.
Manual de usuario (LaTeX y PDF)	Instalación, recorrido, operación paso a paso, ejercicios, solución de problemas y glosario.
Informe final (LaTeX y PDF)	Objetivos, alcance, solución, evaluación, resultados, calidad, riesgos y recomendaciones.

Los tres fuentes LaTeX son autónomos y reproducibles con un motor XeLaTeX moderno o Tectonic, sujeto a disponibilidad de los paquetes estándar declarados.

13 Recomendaciones

13.1 Corto plazo

1. eliminar la inserción HTML no escapada; preferir `textContent` o escapar el mensaje completo de bitácora;
2. añadir pruebas unitarias para tamaños, selección, fusión y métricas;
3. cubrir los casos límite de mensajes y compactación sin espacio libre;
4. validar/ajustar la tabla variable al cambiar la reserva del S.O.;
5. documentar expresamente la convención de fragmentación externa estática.

13.2 Mediano plazo

1. extraer el dominio a un módulo JavaScript puro y dejar el DOM como adaptador;
2. incorporar una suite end-to-end con escenarios reproducibles;
3. permitir exportar/importar configuración, trabajos y bitácora;
4. incluir un modo de comparación lado a lado entre algoritmos;
5. ejecutar auditorías automatizadas de accesibilidad y compatibilidad móvil.

13.3 Extensiones académicas

- siguiente ajuste (*next fit*) y listas libres ordenadas;
- paginación, tablas de páginas y fallos de página;
- segmentación con protección y compartición;
- costo temporal de compactación y reubicación;
- generación de cargas aleatorias con métricas comparativas.

14 Integridad del código fuente

Antes de producir los documentos se registraron las siguientes huellas:

Archivo	SHA-256 inicial
README.md	b00fcb9c6884db5620871af1cc4c5b36 fcec4d064e7b4008b878589c7f9e131a
index.html	67277c06878c8ee2f17b20d6b2a92f5c a0de73367ba3dcb3cf4e67f600e6b37c
css/style.css	083eaae30a30cc87688f7eb674c7a4732 613a009e279b7b18eb6c2097c7c79f3
js/script.js	70cead1c39347565160049b735fcca059 16dbcf603c264c4ccb0e9b61f88f3ba

Estas huellas sirven para comparar el estado posterior y demostrar que la tarea de documentación creó únicamente los entregables solicitados.

15 Conclusión final

Conclusión

El proyecto está completo y coherente como simulador educativo estático de asignación contigua. Su principal fortaleza es convertir cada operación en una representación visual y una explicación rastreable. La base técnica es pequeña, portable y suficientemente clara para enseñanza. Antes de una publicación abierta o una ampliación funcional conviene corregir el manejo HTML de la bitácora, formalizar pruebas y separar el dominio de la capa de presentación. Los manuales y este informe dejan documentado tanto el funcionamiento actual como la ruta de evolución recomendada.